Setup and Basics of Git & GitHub

1) Introduction: What are Git and GitHub?

When writing code, you will make mistakes, change features, and collaborate with others.

- **Git** is a local version control system. It acts like a time machine for your code, tracking every change you make on your computer so you can always revert to a previous, working version.
- **GitHub** is a cloud-based hosting platform for Git repositories. It is essentially a social network and cloud storage for code, allowing you to back up your projects online and collaborate with developers worldwide.

2) Installing Git

1. Go to the official website: git-scm.com/downloads.
2. Download the installer for your operating system.
3. Run the installer. You will see *many* configuration pages—don't worry, you can just click **"Next"** through all of them to accept the default settings.

3) Creating a GitHub Account

1. Go to github.com and click on **"Sign up"** in the top right corner.
2. Follow the prompts: enter your email address, create a strong password, and choose a unique username.
3. Verify your account via the email GitHub sends you.

4) Connecting Git to GitHub (Initial Configuration)

Before you start tracking code, you need to tell your local Git installation who you are so it can label your saved changes correctly. Open your computer's terminal (Command Prompt on Windows, Terminal on Mac) and type the following two commands, replacing the placeholder text with your actual GitHub details:

1. git config --global user.name "Your GitHub Username"
2. git config --global user.email "your.email@example.com"

5) The Most Important Git Functions Explained

To understand how Git works, it helps to know its four main commands, which make up your daily workflow.

- **git clone**: Think of this as "downloading." You use this command to copy an existing project from GitHub onto your local computer so you can work on it.
- **git add (Staging):** When you change a file, Git notices, but it doesn't save it yet. git add tells Git, *"I want to include these specific file changes in my next save."* This is called the "staging area."
- **git commit (Saving):** This permanently saves your staged changes into your local Git history. You always add a short message to explain what you changed (e.g., git commit -m "Fixed login bug").
- **git push (Uploading):** This takes your local, committed saves and uploads them to the cloud on GitHub, ensuring your online backup is up to date and your teammates can see your work.
- **git pull (Updating):** If you are working with others and they upload changes to GitHub, you use git pull to fetch their updates and merge them into your local files.